

School of Management Studies
Motilal Nehru National Institute of Technology Allahabad
Prayagraj – 211004, India

Faculty Incharge,
School of Non-Formal and Continuing Education (SNFCE)
Motilal Nehru National Institute of Technology Allahabad, Prayagraj, India.

Through Head, SMS

Subject: Submission of Summer Internship Report of Mr. Alok Pandey

Dear Sir,

With reference to the letter issued by the School of Non-Formal and Continuing Education (SNFCE), this is to certify that **Mr. Alok Pandey**, a student of **United College of Engineering and Research (UCER), Prayagraj**, has successfully completed his Summer Internship under my supervision in the School of Management Studies, Motilal Nehru National Institute of Technology (MNNIT) Allahabad.

The internship was carried out during the period from **03 June 2026 to 30 June 2026**. During this period, the student actively worked on the research entitled "*Robustness Analysis of an Andes Hantavirus Transmission Network*" and successfully fulfilled all the objectives of the internship.

I am satisfied with his performance, dedication, and technical understanding throughout the training period. Therefore, I kindly request you to accept his internship report and issue the Summer Internship Certificate.

Thanking You.


03/07/2026

Dr. Prabal Pratap Singh

Assistant Professor

Department of School of Management Studies

Motilal Nehru National Institute of Technology Allahabad

Forwarded
Subj: 6/7/26
(officially)

प्रमुख एवं विभागाध्यक्ष/Professor & Head
अध्यक्ष, विभाग/School of Management Studies
मोतीलाल नेहरू राष्ट्रीय प्रौद्योगिकी संस्थान
Motilal Nehru National Institute of Technology
Prayagraj-211004 (UP) India

INTERNSHIP REPORT

on

Robustness Analysis of an Andes Hantavirus Transmission Network

Submitted by
ALOK PANDEY

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering

Under the supervision of

Dr. Prabal Pratap Singh

Assistant Professor
Department of School of Management Studies

Internship Duration:

3rd June – 30th June, 2026



School of Management Studies
Motilal Nehru National Institute of Technology, Prayagraj

June 2026

Undertaking

I hereby declare that the internship report entitled "Robustness Analysis of an Andes Hantavirus Transmission Network", submitted to the Department of School of Management Studies, Motilal Nehru National Institute of Technology (MNNIT) Allahabad, Prayagraj, as a part of the Summer Internship Programme, is my original work carried out under the guidance of Dr. Prabal Pratap Singh.

I further declare that this report has not been submitted, either in whole or in part, to any other institution or university for the award of any degree, diploma, or any other academic qualification. I also confirm that the work presented in this report is original, and wherever the work of others has been used, proper acknowledgement has been given through appropriate references.

Place: Prayagraj

Alok Pandey

Date: _____

CERTIFICATE

I hereby certify that the internship report entitled "*Robustness Analysis of an Andes Hantavirus Transmission Network*" submitted by **Alok Pandey**, a student of **Bachelor of Technology (Computer Science and Engineering)**, United College of Engineering and Research (UCER), Prayagraj, has been carried out under my guidance during the Summer Internship conducted at the Department of School of Management Studies, Motilal Nehru National Institute of Technology (MNNIT), Allahabad.

The work presented in this report is original to the best of my knowledge and has not been submitted elsewhere for the award of any degree, diploma, or other academic qualification.



03/07/2026

Dr. Prabal Pratap Singh

Assistant Professor

Department of School of Management Studies

Motilal Nehru National Institute of Technology

Allahabad - 211004

Date: _____

Acknowledgement

I express my deepest respect and sincere gratitude to **Dr. Prabal Pratap Singh**, my mentor, for his invaluable guidance, constant encouragement, and inspiring mentorship throughout the Summer Internship Program. His expertise, insightful suggestions, and unwavering support were instrumental in broadening my knowledge and successfully completing this work.

I am grateful to my seniors, **Mr. Piyush Rai**, **Mr. Saurabh Kumar**, and **Mr. Adarsh Pratap Singh**, students of **MNNIT Allahabad**, for their guidance, encouragement, and support in helping me secure this internship opportunity.

I also extend my sincere thanks to my friend, **Sahil Pandey**, whose companionship and encouragement made this internship a more enjoyable and enriching learning experience.

I am deeply thankful to my parents for their unconditional love, trust, and unwavering support, and to my parent college for encouraging me to participate in this internship.

I sincerely thank the **School of Management Studies (SMS)** and the **School of Non-Formal & Continuing Education (SNFCE)**, **MNNIT Allahabad**, for providing an excellent platform for learning and research.

Finally, I extend my heartfelt gratitude to everyone who contributed, directly or indirectly, to the successful completion of this internship and this report.

ABSTRACT

This report presents a study on the **Robustness Analysis of Hantavirus Transmission Networks** using graph-based computational techniques. Hantavirus is a zoonotic disease transmitted primarily through infected rodents, and understanding its transmission dynamics is essential for effective disease surveillance and control.

The study begins with an overview of Hantavirus, including its transmission mechanisms, symptoms, and public health significance. It then explores how **graph and network analysis** can be used to model epidemic spread by representing individuals or locations as nodes and their interactions as edges. Such network-based models help identify transmission pathways, influential spreaders, and structural properties that affect disease propagation.

The primary focus of this work is the **Robustness Analysis of The Transmission Network**. Using Python, Pandas, and NetworkX, the network was analyzed under different node removal strategies to evaluate its resilience. Both random failures and targeted removal of highly connected or influential nodes were examined to study their impact on network connectivity and disease transmission. The findings demonstrate that targeted removal of critical nodes significantly reduces network connectivity compared to random removal, highlighting the importance of identifying key transmission points for effective epidemic intervention.

Overall, this study demonstrates that graph-theoretic approaches and network analysis provide valuable insights into understanding epidemic transmission dynamics and assessing the resilience of disease spread networks. The methodologies and findings presented in this report contribute to the application of computational techniques in epidemiology and support the development of more effective disease monitoring and control strategies.

Keywords: Hantavirus, Epidemic Networks, Graph Theory, Network Analysis, Robustness Analysis, Python, Pandas, NetworkX.

Contents

1	Introduction	9
2	Python, Pandas, Graph theory & NetworkX	10
2.1	Python	10
2.1.1	Variable	11
2.1.2	Data type	12
2.1.3	Loops	15
2.1.4	Functions	17
2.2	Pandas	20
2.2.1	Reading csv file	22
2.2.2	Data frames	23
2.2.3	Filtering	25
2.2.4	Filtering	25
2.2.5	Summary statistics	27
2.3	Graph	29
2.3.1	Nodes	30
2.3.2	Edges	31
2.3.3	Directed Graph	32
2.4	NetworkX	33
2.4.1	Creating graphs	35
2.4.2	Adding Nodes	35
2.4.3	Adding Edges	35
2.4.4	Reading Edge Lists	36
2.4.5	Network metrics	36

3	NetworkX for Epidemic Modeling	39
3.1	Introduction	39
3.2	Role of NetworkX in Epidemic Modeling	40
3.2.1	Graph-Based Representation of Epidemics	40
3.2.2	Why NetworkX is Used in Epidemiology	40
3.2.3	Advantages of NetworkX for Epidemic Analysis	40
3.3	Transmission Networks	41
3.3.1	Concept of a Transmission Network	41
3.3.2	Components of a Transmission Network	41
3.3.3	Types of Transmission Networks	42
3.3.4	Applications in Epidemic Modeling	42
3.4	Construction of Epidemic Networks	42
3.4.1	Creating Directed Graphs	42
3.4.2	Adding Nodes and Edges	43
3.4.3	Reading Epidemic Data	44
3.4.4	Visualizing the Network	44
4	Case Study: Modeling the Andes Hantavirus Outbreak	46
4.1	Introduction	46
4.1.1	Background of the Andes Hantavirus	46
4.1.2	Objectives of the Case Study	48
4.2	Outbreak Dataset	50
4.2.1	Transmission Data	50
4.2.2	Preparing the Dataset	53
4.3	Network Robustness Analysis	56
4.3.1	Baseline Network Statistics	56
4.3.2	Random Node Removal	59
4.3.3	Degree-Based Node Removal	62
4.3.4	Betweenness Centrality-Based Node Removal	65
4.3.5	Comparison of Node Removal Strategies	68
4.4	Discussion	70
4.4.1	Key Findings	70
4.4.2	Limitations of the Study	72

4.4.3 Future Scope	73
5 Conclusion	75
References	77

Chapter 1

Introduction

Infectious diseases remain a major global public health challenge, making it essential to understand and control their spread. Network science and graph theory provide effective computational methods for modeling disease transmission and identifying key factors that influence epidemic spread.

This report focuses on the transmission network of **Hantavirus**, a zoonotic disease mainly transmitted from infected rodents to humans. Although human-to-human transmission is rare for most Hantavirus species, studying its transmission network helps improve disease monitoring and intervention strategies.

The network is modeled and analyzed using the Python programming language and the **NetworkX** library. Important network properties, including degree distribution, shortest paths, connected components, clustering coefficients, and centrality measures, are evaluated.

The main objective is to perform a **robustness analysis** by examining how network connectivity changes under random failures and targeted node removal. This helps identify critical nodes responsible for disease spread and demonstrates the value of graph-based methods in epidemic analysis.

The report covers epidemics, Hantavirus, Python and NetworkX-based network analysis, and robustness analysis of transmission networks. ““

Chapter 2

Python, Pandas, Graph theory & NetworkX

2.1 Python

Python is a high-level, interpreted, and general-purpose programming language known for its simplicity, readability, and rapid application development. Developed by Guido van Rossum and released in 1991, it has become one of the most widely used programming languages in academia, research, and industry.

Python supports procedural, object-oriented, and functional programming paradigms, providing flexibility for diverse software applications. It is platform-independent and can run on operating systems such as Windows, Linux, and macOS with minimal modifications.

Its extensive ecosystem of libraries makes Python suitable for scientific computing, data analysis, web development, artificial intelligence, and machine learning. Popular libraries include `NumPy`, `Pandas`, `Matplotlib`, `SciPy`, `Scikit-learn`, `TensorFlow`, and `PyTorch`, while frameworks such as `Django` and `Flask` support web application development.

Due to its versatility, ease of learning, and powerful capabilities, Python is widely used in software development, automation, cybersecurity, cloud computing, image processing, data science, and research.

2.1.1 Variable

A variable in Python is a symbolic name used to store and access data during program execution. It acts as a reference to an object in memory, allowing values to be modified and reused efficiently, thereby improving code readability and maintainability.

Python is a dynamically typed language, meaning the data type of a variable is determined automatically at runtime based on the assigned value. As a result, explicit type declaration is not required.

Variable Declaration

Variables are created by assigning a value using the assignment operator (=). The general syntax is:

```
variable_name = value
```

Characteristics of Variables

Python variables possess several important characteristics:

- Variables do not require explicit data type declarations.
- A variable can store values of different data types during program execution.

Rules for Naming Variables

Python follows specific rules for naming variables:

““latex

- A variable name must begin with a letter or an underscore (_); it cannot start with a digit.
- Only letters, digits, and underscores are permitted; spaces and special characters are not allowed.

““

Examples:

```
student_name = "Rahul"    # Valid
1student = "Rahul"        # Invalid
```

Multiple Variable Assignment

Python allows multiple variables to be assigned values in a single statement.

Assigning different values:

```
x, y, z = 10, 20, 30
```

Assigning the same value:

```
a = b = c = 100
```

Variable Reassignment

Since Python is dynamically typed, the value and data type of a variable can be changed during program execution.

```
x = 25  
print(x)
```

```
x = "Python"  
print(x)
```

Here, the variable `x` first stores an integer and is later reassigned a string value.

2.1.2 Data type

Data types in Python define the kind of value a variable stores and the operations that can be performed on it. Every object has an associated data type, allowing the interpreter to manage memory and execute operations efficiently. Since Python is dynamically typed, the data type is determined automatically from the assigned value.

Python provides several built-in data types, including numeric, sequence, set, mapping, Boolean, binary, and special types, enabling efficient representation and manipulation of different kinds of data.

Numeric Data Types

Numeric data types are used to represent numerical values. Python supports three primary numeric types:

Integer (`int`)

The `int` data type represents whole numbers without a fractional part. Integers can be positive, negative, or zero.

Floating-Point Number (`float`)

The `float` data type represents real numbers containing decimal points or values expressed in scientific notation.

```
“latex
```

Complex Number (`complex`)

Complex numbers contain real and imaginary parts, represented using the suffix `j`.

Sequence Data Types

Sequence data types store ordered collections of elements.

String (`str`)

Strings are immutable sequences of Unicode characters.

List (`list`)

Lists are ordered, mutable collections that can store different data types.

Tuple (`tuple`)

Tuples are ordered collections whose elements cannot be modified after creation.

Set Data Types

Sets store unique, unordered elements. A `frozenset` is the immutable form of a set.

Mapping Data Type

Dictionary (`dict`)

Dictionaries store data as key-value pairs for efficient retrieval.

```
student = {"Name": "Rahul", "Age": 21, "CGPA": 8.5}
```

Boolean Data Type

Boolean variables have only two values: `True` and `False`.

Binary Data Types

Binary data types store data in byte format.

Bytes (bytes)

`bytes` represents immutable byte sequences.

Bytearray (bytearray)

`bytearray` represents mutable byte sequences. “

“`latex id=`”s9kq4m”

Memoryview (memoryview)

A `memoryview` provides direct access to binary data without copying it.

Special Data Type

None Type (NoneType)

`None` represents the absence of a value and is commonly used as a default or null value.

Summary of Python Data Types

Category	Data Types
Numeric	int, float, complex
Sequence	str, list, tuple
Set	set, frozenset
Mapping	dict
Boolean	bool
Binary	bytes, bytearray, memoryview
Special	NoneType

“

“`latex id=`”r8v2nk”

Data Structures

Data structures organize and store data efficiently for easy access and manipulation. Python provides four main built-in data structures: **Lists**, **Tuples**, **Sets**, and **Dictionaries**.

- **List**: Ordered, mutable collection that allows duplicates.
- **Tuple**: Ordered, immutable collection that allows duplicates.
- **Set**: Unordered collection of unique elements.
- **Dictionary**: Stores data as key–value pairs with unique keys.

Data Structure	Ordered	Mutable	Duplicates
List	Yes	Yes	Allowed
Tuple	Yes	No	Allowed
Set	No	Yes	Not Allowed
Dictionary	Yes	Yes	Keys Unique

““

““latex id=”m7q4zp”

2.1.3 Loops

Loops execute a block of code repeatedly, improving program efficiency and reducing code duplication. Python provides two looping constructs:

- **for Loop**
- **while Loop**

The for Loop

The **for** loop iterates over sequences such as lists, tuples, strings, sets, dictionaries, or ranges.

Syntax:

```
for variable in sequence:  
    statements
```

Example:

```
fruits = ["Apple", "Banana", "Mango"]
```

```
for fruit in fruits:  
    print(fruit)
```

Output:

```
Apple  
Banana  
Mango
```

““

““latex id="l2n8qy" The loop executes repeatedly until all values in the sequence are processed.

The while Loop

The while loop executes as long as a specified condition is true.

```
while condition:  
    statements  
  
count = 1  
while count <= 5:  
    print(count)  
    count += 1
```

Loop Control Statements

Python provides the following loop control statements:

- **break:** Terminates the loop.
- **continue:** Skips the current iteration.
- **pass:** Placeholder for an empty loop or block.

Nested Loops

A nested loop is a loop inside another loop and is commonly used for matrix operations and pattern generation.

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, j)
```

““

Difference Between for and while Loops

for Loop	while Loop
Used when the number of iterations is known.	Used when the number of iterations is unknown.
Iterates over a sequence or iterable object.	Executes based on a logical condition.
Simpler and less prone to errors.	Requires manual updating of the loop variable.
Commonly used for traversing collections.	Commonly used for condition-controlled repetition.

2.1.4 Functions

Functions are reusable blocks of code designed to perform specific tasks. They help avoid repetition by allowing code to be written once and executed whenever needed, improving modularity, readability, and maintainability.

Functions make large programs easier to manage by dividing them into smaller modules, simplifying debugging, testing, and maintenance. They are widely used in software development, data science, machine learning, web development, and automation.

Python supports two main types of functions:

- Built-in Functions
- User-defined Functions

Built-in Functions

Built-in functions are predefined functions in Python used for common operations. They are readily available and require no additional implementation.

Common examples include:

- `print()` – Displays output.
- `input()` – Takes user input.
- `len()` – Returns length of an object.

““

““latex id=”k4x8pn”

User-defined Functions

User-defined functions are created using the `def` keyword to perform specific tasks and improve code reusability.

```
def function_name(parameters):  
    statements  
    return value
```

Functions with Parameters and Return Values

Functions can accept input through parameters and return results using the `return` statement.

```
def add(a, b):  
    return a + b  
  
print(add(15, 20))
```

Types of Arguments

Python supports the following argument types:

- **Positional Arguments**

- **Keyword Arguments**
- **Default Arguments**
- **Variable-Length Arguments (*args)**

Lambda Functions

Lambda functions are anonymous functions used for simple one-line operations.

```
square = lambda x: x * x  
print(square(6))
```

““

Local Variables

Variables declared inside a function are called local variables and can only be accessed within that function.

```
def demo():  
    x = 10  
    print(x)
```

```
demo()
```

Global Variables

Variables declared outside all functions are called global variables and can be accessed throughout the program.

```
x = 100
```

```
def display():  
    print(x)
```

```
display()
```

2.2.1 Reading csv file

A Comma-Separated Values (CSV) file is a widely used format for storing tabular data. Each row represents a record and each column represents a field or attribute. Its simplicity and compatibility make it a standard format in data science, machine learning, and data analysis applications. “

Some important benefits include:

- Fast and efficient loading of large datasets.
- Automatic creation of a DataFrame.
- Support for various delimiters and file encodings.

“`latex id="n5r8qx"`

Viewing the Dataset

Pandas provides several functions to inspect a dataset after loading it.

- `df.head()` – Displays the first five rows.
- `df.tail()` – Displays the last five rows.
- `df.info()` – Displays dataset structure and data types.
- `df.describe()` – Provides statistical summary of numerical data.

Important Parameters of `read_csv()`

Commonly used parameters of `read_csv()` include:

- `nrows` – Reads a specified number of rows.
- `usecols` – Selects specific columns.
- `sep` – Specifies the delimiter.
- `header=None` – Reads data without column names.

“

Applications of Reading CSV Files

CSV files are widely used for importing datasets in data analysis, machine learning, and business analytics. “

Advantages of Using `read_csv()`

The `read_csv()` function offers several benefits:

- Simple and user-friendly syntax.
- High performance when reading large datasets.
- Automatic detection of data types.

2.2.2 Data frames

A **DataFrame** is the primary data structure in Pandas used for storing and manipulating structured data. It is a two-dimensional, labeled structure organized in rows and columns, similar to a spreadsheet, SQL table, or database.

Each column in a DataFrame can store different data types such as integers, floats, strings, or dates, making it suitable for real-world datasets. It is built on NumPy and provides efficient tools for data cleaning, transformation, analysis, and computation, making it widely used in data science, machine learning, and analytics. “

Characteristics of a DataFrame

A DataFrame possesses several important characteristics that make it suitable for data analysis:

- It is a two-dimensional data structure consisting of rows and columns.
- Each column can contain different data types.
- Both rows and columns are labeled, enabling efficient indexing.

2.2 Pandas

Pandas is a powerful, open-source Python library used for data manipulation, analysis, and processing. It provides high-performance, easy-to-use data structures and data analysis tools that enable users to efficiently work with structured and tabular data. Developed by Wes McKinney in 2008, Pandas has become one of the most essential libraries in the Python ecosystem and is widely used in data science, machine learning, business analytics, finance, scientific research, and artificial intelligence.

The name **Pandas** is derived from the term "*Panel Data*", referring to multidimensional structured datasets commonly used in statistics and econometrics. Built on top of the NumPy library, Pandas combines the computational efficiency of NumPy with intuitive data structures, making complex data analysis tasks significantly simpler and more efficient.

“`latex id="q7m2vk"`”

Need for Pandas

Pandas is a Python library for efficient data manipulation and analysis. It provides optimized tools for handling large datasets, missing values, filtering, statistical analysis, and file operations.

Core Data Structures in Pandas

Pandas provides two main data structures: **Series** and **DataFrame**.

Series

A **Series** is a one-dimensional labeled array.

```
import pandas as pd
marks = pd.Series([85, 90, 78, 92])
```

DataFrame

A **DataFrame** is a two-dimensional table with rows and columns that supports efficient data storage and analysis.

```
import pandas as pd

student = {
    "Name": ["Alice", "Bob"],
    "Age": [21, 22]
}

df = pd.DataFrame(student)
```

Major Features of Pandas

- Fast and efficient data manipulation.
- Handling missing values.
- Indexing and data selection.
- Filtering and sorting.

Common Operations

- Import datasets.
- Select rows and columns.
- Filter records.
- Display dataset information.

Applications of Pandas

- Data Science
- Machine Learning
- Artificial Intelligence
- Financial Data Analysis

““

Creating a DataFrame

A DataFrame can be created from various data sources, including dictionaries, lists, NumPy arrays, CSV files, Excel files, SQL databases, and JSON files.

The general syntax is:

```
import pandas as pd

df = pd.DataFrame(data)
```

“`latex id="p4x7zn"`”

Accessing Data in a DataFrame

Data in a DataFrame can be accessed using column names, index-based indexing (`iloc`), or label-based indexing (`loc`).

```
print(df["Name"])
print(df.iloc[0])
```

Common DataFrame Operations

Common DataFrame functions include:

- `df.head()` – Displays first rows.
- `df.info()` – Shows dataset structure.
- `df.describe()` – Generates statistical summary.
- `df.shape` – Returns dataset dimensions.

Modifying a DataFrame

Columns can be added, updated, or removed.

```
df["Grade"] = ["A", "A+", "B+"]
df = df.drop("Grade", axis=1)
```

Handling Missing Values

Missing values can be detected, removed, or replaced using:

- `isnull()`
- `dropna()`
- `fillna()`

Sorting and Filtering Data

Data can be sorted using `sort_values()` and filtered using conditional expressions. “

2.2.3 Filtering

Filtering in Pandas refers to selecting specific rows or columns from a DataFrame based on given conditions. It helps extract only relevant data instead of processing the entire dataset, making analysis more efficient and focused.

In real-world applications, filtering is essential for handling large datasets and is widely used in data science, machine learning, business analytics, and research for extracting meaningful subsets of data.

Need for Filtering

Filtering is used to retrieve data that satisfies specific conditions, reduce unnecessary information, and improve analysis efficiency.

- Selecting records based on conditions.
- Removing irrelevant or unwanted data.
- Preparing datasets for analysis and machine learning.

“

“`latex id="f8q2mv"`

2.2.4 Filtering

Filtering in Pandas selects rows or columns that satisfy specified conditions, making data analysis more efficient.

Need for Filtering

Filtering is used to:

- Select records based on conditions.
- Remove irrelevant data.
- Prepare datasets for analysis.

Need for Data Cleaning

Real-world datasets often contain missing, duplicate, or inconsistent data. Data cleaning improves data quality and ensures reliable analysis.

- Remove incorrect and duplicate records.
- Handle missing values.
- Improve the accuracy of analysis.

““

Handling Missing Values

Missing values are represented as `NaN`. They can be detected, removed, or replaced using `isnull()`, `dropna()`, and `fillna()`.

Data Cleaning Operations

Common data cleaning functions include:

- `drop_duplicates()` – Remove duplicate rows.
- `rename()` – Rename columns.
- `astype()` – Convert data types.
- `drop()` – Remove rows or columns.
- `replace()` – Replace specific values.

Sorting Data

Data can be sorted using `sort_values()` in ascending or descending order.

Handling Outliers

Outliers are values that differ significantly from other observations. They can be identified using filtering techniques to improve data analysis. “

2.2.5 Summary statistics

Summary statistics are numerical measures that describe the main characteristics of a dataset, including its central tendency, variability, and distribution. They are a key part of Exploratory Data Analysis (EDA) and are widely used in data science, machine learning, and business analytics.

Pandas provides built-in functions to generate statistical summaries for numerical and categorical data, helping in quick identification of trends, anomalies, and data quality issues.

Need for Summary Statistics

For large datasets, manual inspection is not practical. Summary statistics condense data into meaningful measures that support quick analysis.

- Understanding overall dataset characteristics.
- Identifying patterns and trends.
- Detecting missing values and outliers.

The `describe()` Function

The most commonly used function for generating summary statistics in Pandas is `describe()`.

`df.describe()`

The `describe()` function provides summary statistics for numerical columns in a dataset.

Measures Included in Summary Statistics

Measure	Description
Count	Number of non-missing values.
Mean	Arithmetic average of the data.
Median	Middle value of the sorted data.
Mode	Most frequently occurring value.
Standard Deviation	Measures the spread of data around the mean.
Variance	Measures the variability of data.
Minimum	Smallest value in the dataset.
Maximum	Largest value in the dataset.

““

Summary Statistics for Categorical Data

Summary statistics can also be generated for categorical variables.

```
df.describe(include="object")
```

This displays information such as:

- Total number of entries.
- Number of unique values.
- Most frequently occurring value.
- Frequency of the most common value.

2.3 Graph

Graphs are an effective way to visualize data and identify patterns, trends, relationships, and outliers more easily than numerical representations. Data visualization is widely used in data analysis, machine learning, business intelligence, and scientific research for better interpretation of results.

In Python, `Matplotlib` is the most commonly used library for data visualization. It is an open-source library that provides various plotting functions for creating high-quality graphs and integrates well with libraries such as NumPy and Pandas.

Introduction to Matplotlib

Matplotlib is a powerful Python library used for creating static, animated, and interactive visualizations. The most commonly used module is `matplotlib.pyplot`, which provides functions for generating different types of graphs.

It is imported as:

```
import matplotlib.pyplot as plt
```

The alias `plt` simplifies the use of plotting functions in Python programs.

Bar Graph

A bar graph compares numerical values across different categories using rectangular bars.

Histogram

A histogram illustrates the frequency distribution of numerical data by dividing values into intervals called bins.

Scatter Plot

A scatter plot displays the relationship between two numerical variables using individual points.

Pie Chart

A pie chart represents the proportional contribution of different categories to a whole.

“`latex id=`”v6n3kp”

Adding Labels and Titles

Titles and axis labels improve the readability of graphs using `title()`, `xlabel()`, and `ylabel()`.

Displaying a Grid

The `grid()` function improves graph readability.

Saving a Graph

Graphs can be saved as image files using `savefig()`.

Advantages of Graphs

- Simplify data interpretation.
- Highlight trends and patterns.
- Enable comparison between variables.

Applications of Graphs

Graphs are widely used in:

- Data science and machine learning.
- Business analytics.
- Financial analysis.
- Scientific research.

““

2.3.1 Nodes

A **node** is the basic unit of a data structure that stores data and one or more references to other nodes. Nodes are the building blocks of linked lists, trees, graphs, stacks, and queues.

Nodes are dynamically allocated and connected through references, making them suitable for dynamic data structures. In Python, nodes are typically implemented using classes.

Structure of a Node

A node consists of:

- **Data:** Stores the value.
- **Reference (Link):** Connects to another node.

““

Types and Applications of Nodes

Nodes are classified based on their structure and use. Common types include single nodes, double nodes, tree nodes, and graph nodes. They are widely used in linked lists, trees, graphs, stacks, queues, database indexing, memory management, and social network analysis.

2.3.2 Edges

An **edge** is a connection between two nodes that represents a relationship between them. Edges may be:

- **Undirected:** Bidirectional connection ($A \leftrightarrow B$).
- **Directed:** One-way connection ($A \rightarrow B$).
- **Weighted:** Associated with a cost, distance, or capacity.

““

““

Weighted edges are used in shortest-path and optimization problems.

““`latex`

Creating Edges Using NetworkX

Edges can be added to a graph using the `add_edge()` function.

```
import networkx as nx
```

```
G = nx.Graph()
```

```
G.add_edge("A", "B")
```

Edge Operations

Common edge operations include adding, removing, and checking edges, as well as computing shortest paths.

Applications

Edges are widely used in transportation, communication, social networks, recommendation systems, and biological networks.

Difference Between Nodes and Edges

Node (Vertex)	Edge
Represents an entity.	Represents a relationship.
Stores data.	Connects two nodes.
Example: Person, City.	Example: Road, Friendship.

““

“\latex

2.3.3 Directed Graph

A **Directed Graph (Digraph)** is a graph in which each edge has a direction, representing a one-way relationship between nodes. It is commonly used to model web links, workflows, and transportation networks.

A directed graph is represented as:

$$G = (V, E)$$

where V is the set of vertices and E is the set of directed edges.

Characteristics

- Edges have direction.
- Represented as ordered pairs.
- Supports in-degree and out-degree.

NetworkX Representation

```
import networkx as nx
```

```
G = nx.DiGraph()
```

```
G.add_edge("A", "B")
```

Applications

Directed graphs are widely used in web search engines, social networks, transportation systems, project scheduling, network routing, and workflow analysis.

Difference Between Directed and Undirected Graphs

Directed Graph	Undirected Graph
Edges have direction.	Edges have no direction.
One-way connections.	Two-way connections.
Has in-degree and out-degree.	Has only degree.

““

“latex

2.4 NetworkX

NetworkX is an open-source Python library used to create, analyze, and visualize graphs. It supports graph structures such as undirected, directed, and weighted graphs, along with algorithms for network analysis.

Key Features

- Supports multiple graph types.
- Built-in graph algorithms.
- Integration with NumPy, Pandas, and Matplotlib.
- Graph visualization support.

Basic Example

```
import networkx as nx
```

```
G = nx.DiGraph()
```

```
G.add_edge("A", "B")
```

Common Functions

- `add_node()`, `add_edge()`
- `nodes()`, `edges()`
- `degree()`
- `number_of_nodes()`, `number_of_edges()`

Applications

NetworkX is widely used in social network analysis, transportation systems, communication networks, disease transmission modeling, and machine learning.

Advantages and Limitations

NetworkX provides an easy-to-use interface with extensive graph algorithms and visualization support. However, it is less suitable for very large or dense networks because of higher memory usage and lower performance. ““

2.4.1 Creating graphs

A graph is a core data structure in NetworkX used to represent relationships between entities. Before analysis, a graph object must be created to store nodes and edges. NetworkX provides different graph types such as undirected graphs, directed graphs, multigraphs, and multidigraphs depending on the application.

“`latex`

Types of Graphs in NetworkX

NetworkX supports different graph types:

- `Graph()` – Undirected graph.
- `DiGraph()` – Directed graph.
- `MultiGraph()` – Multiple undirected edges.
- `MultiDiGraph()` – Multiple directed edges.

Applications

NetworkX is widely used in social network analysis, disease transmission modeling, transportation systems, biological networks, and recommendation systems.

2.4.2 Adding Nodes

Nodes represent entities such as people, cities, or patients. They can be added individually or in groups.

```
G.add_node(1)
```

```
G.add_nodes_from([2, 3, 4])
```

Node attributes can also be assigned.

2.4.3 Adding Edges

Edges define relationships between nodes and can be added using:

```
G.add_edge(1, 2)
G.add_edges_from([(2,3), (3,4)])
```

Edge attributes such as weights can also be included.

2.4.4 Reading Edge Lists

An edge list stores graph connections, where each row represents an edge between two nodes.

```
G = nx.read_edgelist("edges.txt")
```

Weighted and directed edge lists are also supported through appropriate NetworkX functions.

Applications

Edge lists are commonly used in social networks, disease transmission networks, transportation systems, communication networks, and biological networks. “

2.4.5 Network metrics

Network metrics are quantitative measures used to analyze the structural properties of a graph. They describe how nodes are connected, how information flows, and which nodes are important. In NetworkX, these metrics are computed using built-in functions and are widely used in social networks, biological systems, transportation, communication networks, and epidemiological studies.

Degree represents the number of edges connected to a node. In directed graphs, it is divided into in-degree and out-degree. It helps identify highly connected nodes.

Degree Centrality measures the importance of a node based on its direct connections.

Betweenness Centrality identifies nodes that lie on shortest paths and act as bridges in the network.

Closeness Centrality measures how close a node is to all other nodes, indicating how quickly it can reach others.

Eigenvector Centrality considers both the number and importance of neighboring nodes.

Clustering Coefficient measures how strongly a node's neighbors are interconnected, indicating local clusters.

Average Shortest Path Length is the mean distance between all node pairs, representing communication efficiency.

Network Density is the ratio of actual edges to possible edges, showing how dense the graph is.

Diameter is the longest shortest path between any two nodes.

Connected Components are groups of nodes connected internally but disconnected from others.

Although each metric provides different insight, they are usually analyzed together to understand overall network structure. These metrics also help compare different networks using standardized numerical values.

Common Network Metrics in NetworkX

NetworkX provides several built-in functions for network metrics:

- Degree
- Degree Centrality
- Betweenness Centrality
- Closeness Centrality
- Eigenvector Centrality
- Clustering Coefficient
- Average Shortest Path Length
- Network Density
- Diameter
- Connected Components

Advantages of Network Metrics

- Describe network structure quantitatively.
- Identify important nodes.
- Measure connectivity and efficiency.
- Detect clusters and communities.
- Compare different networks.
- Support optimization and decision-making.
- Useful in social and epidemiological analysis.

Applications of Network Metrics

- Social network analysis.
- Disease transmission modeling.
- Transportation systems.
- Communication networks.
- Biological networks.
- Network security.
- Recommendation systems.
- Financial networks.
- Infrastructure planning.

Summary: Network metrics convert graph structures into numerical measures that help analyze connectivity, important nodes, and overall network behavior. NetworkX provides efficient tools for this analysis.

Chapter 3

NetworkX for Epidemic Modeling

3.1 Introduction

The spread of infectious diseases is a major global concern. Understanding transmission patterns helps in identifying infection sources, predicting outbreaks, and planning control strategies. Traditional epidemiological methods rely on statistics and tables, but they often fail to capture complex interactions.

Network science addresses this by representing individuals as nodes and transmission events as edges. This graph-based approach helps visualize and analyze disease spread more effectively.

NetworkX is a widely used Python library for building and analyzing such networks. It allows modeling epidemic transmission using graphs and applying graph algorithms for analysis and visualization.

Compared to traditional methods, network-based modeling helps identify transmission paths, key spreaders, infection clusters, and structural properties of outbreaks. It also supports dynamic updates as new data becomes available.

NetworkX supports directed, weighted, and temporal graphs, making it suitable for real-world epidemic modeling.

This chapter introduces the use of NetworkX in epidemic modeling, forming the basis for the case study in the next chapter.

3.2 Role of NetworkX in Epidemic Modeling

3.2.1 Graph-Based Representation of Epidemics

An epidemic spreads through interactions between individuals. Graph theory represents this using nodes (individuals) and edges (transmission events). A directed edge shows the direction of infection from one person to another.

This model helps identify transmission chains, clusters, and key infected individuals, which are difficult to detect using traditional methods.

Graph-based representation also allows the application of network algorithms to analyze outbreak structure and dynamics effectively.

3.2.2 Why NetworkX is Used in Epidemiology

NetworkX is an open-source Python library for creating, analyzing, and visualizing complex networks. It is widely used in epidemiology due to its rich graph algorithms and flexible data structures.

In epidemic modeling, NetworkX converts epidemiological data into graphs. Patients are represented as nodes, transmission events as edges, and attributes like age, location, or transmission probability can be stored in nodes or edges. This allows detailed modeling of real outbreaks.

It supports different graph types such as directed, undirected, weighted, multigraphs, and DAGs. Directed graphs are used for transmission modeling, while weighted graphs represent probability or strength of infection.

NetworkX also provides built-in functions to compute degrees, shortest paths, centrality measures, clustering, and connected components, making epidemic analysis easier and more structured.

3.2.3 Advantages of NetworkX for Epidemic Analysis

NetworkX simplifies epidemic modeling by representing complex transmission data as graphs instead of tables, making analysis easier.

It is computationally efficient and supports fast graph algorithms. New data can be added easily without changing the overall structure.

It also helps identify key spreaders, clusters, and transmission patterns. Integration with NumPy, Pandas, and Matplotlib enables complete workflows for data processing, analysis, and visualization.

Overall, NetworkX provides a complete framework for epidemic modeling with strong support for graph construction, analysis, and visualization.

3.3 Transmission Networks

3.3.1 Concept of a Transmission Network

A transmission network shows how a disease spreads in a population using a graph structure. Individuals are nodes and transmission events are edges.

Unlike tabular data, it clearly shows infection pathways, sources, and clusters. As the epidemic progresses, new nodes and edges are added, allowing real-time tracking of disease spread.

It also enables graph-based analysis such as connectivity, shortest paths, and centrality, providing deeper insights into epidemic dynamics than traditional methods.—————

3.3.2 Components of a Transmission Network

A transmission network consists of basic elements that represent disease spread.

The **node** represents an individual or patient. In epidemiology, each infected person is modeled as a node, with attributes like age, location, or status if needed.

The **edge** represents a transmission or contact between two individuals, forming the infection pathway.

Direction indicates the flow of infection, usually from infected to newly infected individuals, preserving transmission order.

Edge weights may represent transmission probability, contact strength, or confidence level, making the model more realistic.

Together, nodes, edges, direction, and weights define the structure of a transmission network.

3.3.3 Types of Transmission Networks

Transmission networks are classified based on structure and data.

A **directed network** shows infection flow between individuals and is most commonly used in epidemiology.

An **undirected network** ignores direction and is used when infection source is unknown or only contact patterns are studied.

A **weighted network** assigns values to edges such as probability or contact duration.

A **temporal network** includes time information to study how the epidemic evolves.

The choice depends on data availability and research goals.

3.3.4 Applications in Epidemic Modeling

Transmission networks are widely used in epidemiology for multiple purposes.

They support **contact tracing** by identifying infection sources and exposed individuals.

They help detect **clusters and super-spreaders**, enabling targeted control measures.

They are used to estimate key parameters like R_0 and transmission rates.

They also support simulation, prediction, and evaluation of intervention strategies.

Overall, transmission networks combined with tools like NetworkX provide a powerful framework for analyzing disease spread efficiently.

3.4 Construction of Epidemic Networks

3.4.1 Creating Directed Graphs

The first step in epidemic network modeling is to construct a graph that accurately represents the transmission of disease. Since infectious diseases generally spread from one individual to another in a specific direction, directed graphs are commonly used in epidemiological studies. A directed graph preserves the direction of transmission by connecting the source of infection to the newly infected individual through a directed edge.

NetworkX provides the `DiGraph()` class for creating directed graphs. This class allows researchers to build transmission networks while maintaining the chronological order of disease spread. Initially, an empty directed graph is created, and as epidemiological information becomes available, nodes and edges are gradually added to represent infected individuals and transmission events.

Using directed graphs offers several advantages. They clearly identify infection sources, reconstruct transmission pathways, and support graph algorithms specifically designed for directed networks. These properties make directed graphs highly suitable for modeling epidemic outbreaks and analyzing disease dynamics.

Directed graphs also provide the foundation for advanced network analysis, including topological sorting, shortest path analysis, centrality measures, and reproduction number estimation. Consequently, they form the core structure of most graph-based epidemic models.

3.4.2 Adding Nodes and Edges

After creating the directed graph, the next step is to add nodes and edges that represent the epidemic data. In a transmission network, each infected individual is represented as a node, while every confirmed transmission event is represented as a directed edge connecting two nodes.

Each node may contain additional information such as patient identification number, age, gender, location, or clinical status. Similarly, edges may store attributes including transmission date, transmission probability, contact duration, or confidence level. These attributes enrich the network and support more detailed epidemiological analysis.

As new infection records become available, additional nodes and edges can easily be incorporated into the graph. This flexibility allows the transmission network to evolve continuously during an outbreak without requiring reconstruction of the entire graph.

The process of adding nodes and edges transforms raw epidemiological records into a structured network that can be analyzed using the algorithms available in NetworkX. Once completed, the graph provides a comprehensive representation of disease transmission within the affected population.

3.4.3 Reading Epidemic Data

Epidemiological information is commonly stored in structured datasets such as CSV files, Excel spreadsheets, or relational databases. These datasets typically contain patient identifiers, dates of infection, exposure history, contact tracing information, and other variables describing disease transmission.

Before constructing the transmission network, the epidemiological data must be imported into the programming environment and organized into an appropriate format. Libraries such as Pandas are frequently used to read and preprocess these datasets before they are converted into graph structures using NetworkX.

After the data have been cleaned and validated, each record describing a transmission event can be transformed into a node-edge relationship within the graph. This systematic conversion ensures that the resulting transmission network accurately represents the available epidemiological information and minimizes errors caused by missing or inconsistent records.

Proper data preparation is therefore an essential step in epidemic modeling because the accuracy of subsequent network analysis depends directly on the quality of the input data.

3.4.4 Visualizing the Network

Visualization plays an important role in understanding epidemic transmission networks. Instead of examining large datasets containing hundreds or thousands of records, researchers can interpret disease spread through a graphical representation that clearly illustrates relationships among infected individuals.

Network visualization displays nodes as individuals and edges as transmission events, providing an intuitive overview of outbreak progression. Well-designed visualizations make it easier to identify transmission chains, infection clusters, highly connected individuals, and isolated cases that may require further epidemiological investigation.

Graph visualization also assists in validating the network structure before performing advanced computational analyses. By examining the graphical representation, researchers can detect missing nodes, incorrect transmission pathways, or inconsistencies within the dataset.

Furthermore, network visualizations improve communication between researchers, healthcare professionals, and public health authorities. Complex epidemiological information can be presented in a clear and understandable manner, supporting evidence-based decision-making during epidemic response and disease surveillance.

Chapter 4

Case Study: Modeling the Andes Hantavirus Outbreak

4.1 Introduction

4.1.1 Background of the Andes Hantavirus

The Andes Hantavirus (ANDV) is a zoonotic virus belonging to the *Orthohantavirus* genus of the *Hantaviridae* family. It is primarily found in South America, particularly in Argentina and Chile, where it is naturally carried by infected rodents. Human infection usually occurs through exposure to rodent saliva, urine, or feces that contaminate the surrounding environment. Unlike most other hantaviruses, the Andes Hantavirus has demonstrated the ability to spread from one infected person to another under specific circumstances, making it an important subject of epidemiological research.

The disease caused by the Andes Hantavirus is known as **Hantavirus Cardiopulmonary Syndrome (HCPS)**, a severe respiratory illness associated with high mortality. The disease generally begins with non-specific symptoms such as fever, headache, muscle pain, and fatigue before progressing to serious pulmonary complications. Because of its rapid progression and high fatality rate, early identification of transmission pathways is essential for effective outbreak control and public health intervention.

One of the distinguishing characteristics of the Andes Hantavirus is its documented human-to-human transmission. This unique feature differentiates it from many other hantaviruses and increases the complexity of epidemiological investigations. Consequently,

researchers must carefully analyze transmission events to determine how the infection spreads through different generations of individuals and to identify potential transmission clusters.

Traditional epidemiological investigations rely on patient records, contact tracing, laboratory reports, and statistical analyses. Although these approaches provide valuable information, they often fail to clearly illustrate the complex relationships among infected individuals. Graph-based epidemic modeling addresses this limitation by representing individuals and transmission events as interconnected components of a network.

In graph theory, each infected individual is represented as a **node**, while each confirmed transmission event is represented as a **directed edge**. This graphical representation preserves the direction of disease transmission and enables researchers to reconstruct transmission chains accurately. Such network models also facilitate the application of graph algorithms for identifying infection sources, transmission pathways, highly connected individuals, and transmission clusters.

The Python library **NetworkX** provides a comprehensive framework for constructing, analyzing, and visualizing epidemic transmission networks. It supports directed graphs, weighted graphs, graph traversal algorithms, centrality measures, topological analysis, and numerous other computational techniques that are particularly useful in epidemiological research. These capabilities allow researchers to transform raw transmission records into structured networks that can be explored both visually and mathematically.

In this case study, the Andes Hantavirus outbreak is modeled using NetworkX as a directed transmission network. The investigation begins with the construction of the transmission graph from epidemiological records. After validating the network structure, several graph-theoretic analyses are performed, including network visualization, topological ordering, computation of network metrics, estimation of the Basic Reproduction Number (R_0), and robustness analysis. Weighted transmission networks are also introduced to represent uncertainty in transmission events and to improve the realism of the epidemiological model.

The primary objective of this case study is to demonstrate how graph theory and the NetworkX library can be applied to investigate infectious disease outbreaks. By converting epidemiological data into a network structure, researchers can obtain valuable insights into disease transmission dynamics, identify critical transmission pathways, and evaluate

intervention strategies. The methods presented in this chapter illustrate the practical application of computational epidemiology and highlight the importance of network science in supporting evidence-based public health decision-making.

Image Type: Geographic distribution map of the Andes Hantavirus in South America (Argentina and Chile).

Figure Caption:

Figure 5.1: Geographical distribution of the Andes Hantavirus in South America.

4.1.2 Objectives of the Case Study

The primary objective of this case study is to demonstrate the practical application of the NetworkX library in modeling and analyzing an epidemic transmission network. By converting epidemiological records into a directed graph, the study illustrates how graph theory can be used to investigate disease transmission patterns, identify infection pathways, and extract meaningful epidemiological information.

The first objective is to construct a transmission network from the available outbreak data. Each infected individual is represented as a node, while each confirmed transmission event is represented as a directed edge. This graphical representation preserves the direction of disease transmission and provides a clear understanding of how the epidemic progresses from one individual to another.

The second objective is to visualize the transmission network using NetworkX. Visualization enables researchers to observe the overall structure of the outbreak, identify transmission chains, detect clusters of infected individuals, and understand the progression of the disease across multiple generations.

Another important objective is to validate the constructed network and verify that it accurately represents the epidemiological data. Validation ensures that all transmission relationships are correctly modeled before advanced graph analysis is performed.

The study also aims to analyze the transmission network using graph-theoretic techniques. Important analyses include identifying transmission generations through topological ordering, computing network metrics, and evaluating the structural characteristics of the epidemic network.

An additional objective is to estimate the Basic Reproduction Number (R_0) from the transmission network. This parameter provides valuable information regarding the

transmission potential of the disease and helps determine whether the epidemic is likely to expand or decline.

The case study further investigates uncertain transmission events by assigning weights to transmission edges. Weighted transmission networks provide a more realistic representation of epidemic spread because they account for varying levels of confidence associated with different transmission events.

Finally, the study evaluates the robustness of the transmission network under different intervention strategies. By comparing random node removal, degree-based removal, and betweenness-based removal, the effectiveness of different outbreak control approaches can be assessed.

The overall objective of this case study is to demonstrate that NetworkX provides a flexible and efficient computational framework for epidemic modeling. The methods presented in this chapter illustrate how graph theory can support outbreak investigation, disease surveillance, and evidence-based public health decision-making.

Objectives Summary

- Construct a directed transmission network from epidemiological data.
- Represent infected individuals as nodes and transmission events as directed edges.
- Visualize the epidemic transmission network.
- Validate the constructed network.
- Analyze the network using graph algorithms.
- Compute the Basic Reproduction Number (R_0).
- Model uncertain transmission using weighted edges.
- Evaluate network robustness using different intervention strategies.
- Demonstrate the practical application of NetworkX in computational epidemiology.

Workflow of the Case Study

The complete workflow adopted in this case study is illustrated in Figure 4.1. The process begins with collecting epidemiological transmission data, followed by constructing

a directed graph using NetworkX. The constructed network is then validated and analyzed using graph-theoretic techniques. Important epidemiological parameters such as the Basic Reproduction Number (R_0) are computed, after which weighted transmission networks are introduced to model uncertainty. Finally, robustness analysis is performed to evaluate different intervention strategies and assess the resilience of the transmission network.

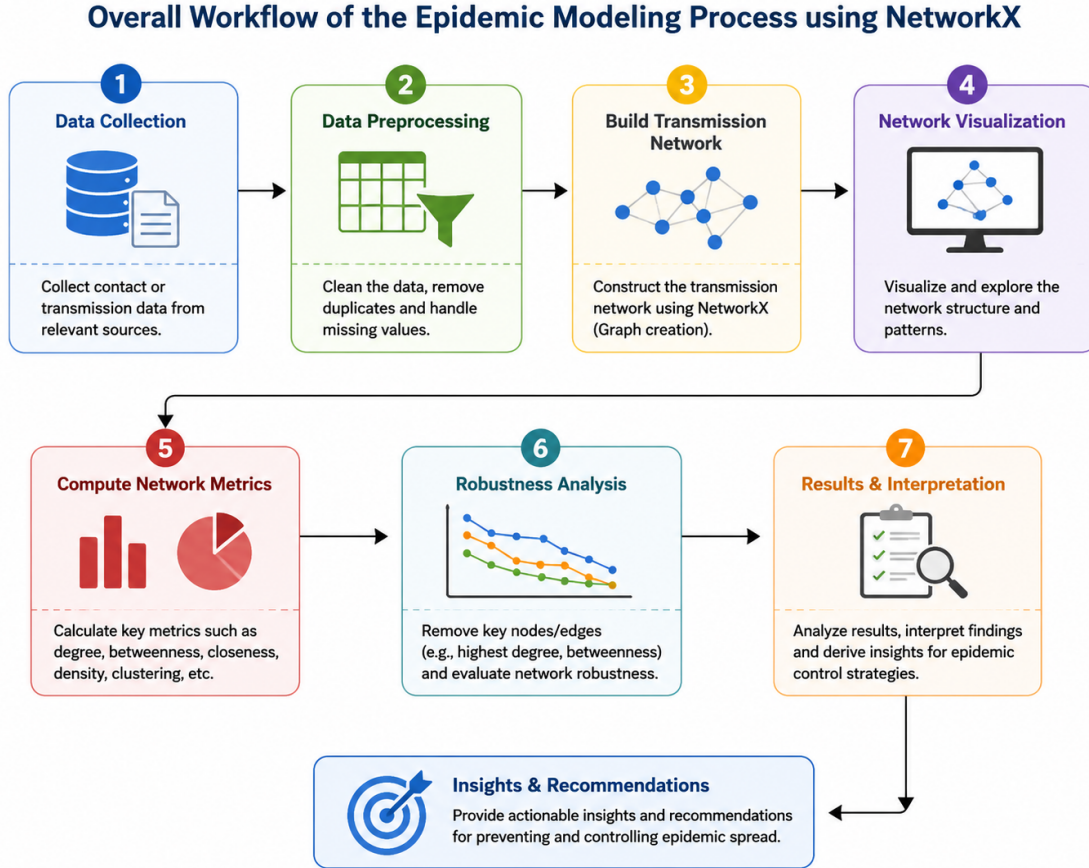


Figure 4.1: Overall workflow of the epidemic modeling process using NetworkX.

4.2 Outbreak Dataset

4.2.1 Transmission Data

The outbreak dataset forms the foundation of the epidemic transmission network. It contains epidemiological records that describe how the disease spread from one infected individual to another during the outbreak. Each record represents a confirmed transmission event obtained through contact tracing, laboratory investigations, and epidemiological surveillance. These records provide the essential information required to reconstruct the

transmission chain using graph theory.

In this case study, the transmission dataset consists of pairs of infected individuals, where the first individual represents the source of infection and the second individual represents the newly infected patient. Such source-target relationships naturally correspond to directed edges in a graph, making the dataset highly suitable for network-based analysis using the NetworkX library.

Before constructing the transmission network, the dataset is carefully examined to ensure that the reported transmission events are complete and logically consistent. Each patient is assigned a unique identifier so that every infected individual can be represented as a distinct node within the graph. Similarly, every confirmed transmission event becomes a directed edge connecting the corresponding source and destination nodes.

The transmission dataset not only records who infected whom but also preserves the chronological order of disease spread. This temporal information enables researchers to reconstruct transmission pathways, identify infection generations, and study the progression of the epidemic over time. Such analyses provide valuable insights into the dynamics of disease transmission and support evidence-based public health decision-making.

The NetworkX library provides an efficient framework for transforming these epidemiological records into a directed graph. After importing the dataset into Python, each transmission pair is converted into a directed edge, resulting in a transmission network that accurately represents the outbreak. Once the graph has been constructed, numerous graph algorithms can be applied to investigate the structural and epidemiological characteristics of the network.

The transmission dataset therefore serves as the primary input for all subsequent analyses performed in this case study, including network visualization, validation, computation of network metrics, estimation of the Basic Reproduction Number (R_0), weighted network analysis, and robustness analysis.

Python Code

```
# Import NetworkX
import networkx as nx

# Create an empty directed graph
```

```
G = nx.DiGraph()

# Example transmission records
transmissions = [
    ("Patient 1", "Patient 2"),
    ("Patient 1", "Patient 3"),
    ("Patient 2", "Patient 4"),
    ("Patient 2", "Patient 5"),
    ("Patient 3", "Patient 6")
]
```

Code Explanation

The above code imports the NetworkX library and creates an empty directed graph using the `DiGraph()` class. A sample transmission dataset is then defined as a list of source-destination pairs, where each tuple represents one confirmed transmission event. These transmission records will later be inserted into the graph to construct the epidemic transmission network.

Expected Output

```
Transmission Dataset Loaded Successfully
```

```
Total Transmission Records : 5
```

```
Unique Patients             : 6
```

```
Graph Type                  : Directed Graph
```

Transmission Dataset

Transmission Dataset Used for Constructing the Epidemic Network					
(Raw / Original Data)					
Source ID	Target ID	Transmission Date	Transmission Type	Location	Remarks / Issues
1	2	2024-01-01	Direct	Household	Valid
1	3	2024-01-02	Direct	Community	Valid
1	4	2024-01-02	Direct	Community	Valid
1	5	2024-01-03	Direct	Household	Valid
2	2	2024-01-03	Household	Household	Self-loop
2	6	2024-01-03	Direct	Community	Valid
2	7	2024-01-04	Household	Household	Valid
2	8	2024-01-04	Household	Household	Valid
3	5	2024-01-03	Community	Community	Duplicate
3	5	2024-01-03	Community	Community	Duplicate
4	9	2024-01-05	Direct	Community	Valid
5	10	2024-01-05	Household	Household	Valid
6	11	2024-01-06	Household	Household	Valid
6	9	2024-01-07	Direct	Community	Valid
7	15	2024-01-07	Community	Community	Valid
7	16	2024-01-08	Community	Community	Valid
9	20	2024-01-09	Community	Community	Valid
9	21	2024-01-09	Household	Household	Valid
10	NA	2024-01-10	Direct	Community	Missing Target
NA	12	2024-01-10	Household	Household	Missing Source
12	31	2024-01-11	Community	Community	Valid
13	13	2024-01-12	Direct	Community	Self-loop
...	...	...	...	...	...

Figure 4.2: Transmission dataset used for constructing the epidemic network.

Interpretation

The transmission dataset represents the epidemiological relationships among infected individuals and serves as the primary input for constructing the epidemic transmission network. After organizing the transmission records into source-target pairs, the dataset becomes suitable for graph-based analysis using NetworkX. The next step is to convert these transmission records into a directed graph by creating nodes and edges that accurately represent the observed disease transmission.

4.2.2 Preparing the Dataset

Before constructing the transmission network, the outbreak dataset must be prepared to ensure that it is complete, accurate, and suitable for graph-based analysis. Epidemiological datasets often contain missing values, duplicate records, inconsistent patient identifiers, or incomplete transmission information. If these issues are not addressed before network construction, they may produce incorrect graph structures and misleading analytical results.

The first step in data preparation is to examine the dataset and verify that each infected individual has a unique identifier. These identifiers allow every patient to be

represented as a distinct node in the transmission network. Similarly, each transmission event should contain both a source patient and a destination patient, ensuring that every directed edge correctly represents the flow of infection.

After verifying the patient identifiers, duplicate transmission records are removed to prevent multiple identical edges from appearing in the graph. Missing values and incomplete transmission events are also identified and handled appropriately. These pre-processing steps improve the reliability of the transmission network and reduce errors during graph analysis.

The cleaned transmission dataset is then organized into a format suitable for NetworkX. Each row of the dataset represents one confirmed transmission event and contains the source patient together with the newly infected patient. This structured representation enables the dataset to be directly converted into a directed graph using the functions provided by the NetworkX library.

Proper dataset preparation is an essential stage of epidemic modeling because every subsequent analysis depends on the quality of the input data. A carefully prepared dataset produces a transmission network that accurately reflects the observed outbreak and supports reliable epidemiological investigations.

Python Code

```
import pandas as pd

# Read transmission dataset
df = pd.read_csv("transmission_data.csv")

# Display first five records
print(df.head())

# Check dataset information
print(df.info())

# Check missing values
print(df.isnull().sum())
```

```
# Remove duplicate records
df = df.drop_duplicates()

# Reset row indices
df.reset_index(drop=True, inplace=True)

print("Dataset prepared successfully.")
```

Code Explanation

The above code imports the Pandas library and reads the outbreak transmission dataset from a CSV file. The first few records are displayed to verify that the data have been loaded correctly. Dataset information and missing values are then examined to identify potential inconsistencies. Duplicate transmission records are removed, and the row indices are reset to produce a clean and organized dataset that is ready for graph construction.

Expected Output

Dataset Information

```
Rows                : XX
Columns             : XX
Missing Values      : 0
Duplicate Records    : Removed
```

```
Dataset prepared successfully.
```

Prepared Transmission Dataset After Preprocessing						
Source ID	Target ID	Source Type	Target Type	Transmission Date	Transmission Type	Validation Status
1	2	Patient	Patient	2024-01-01	Direct	Valid
1	3	Patient	Patient	2024-01-02	Direct	Valid
1	4	Patient	Patient	2024-01-02	Direct	Valid
1	5	Patient	Patient	2024-01-03	Direct	Valid
2	6	Patient	Patient	2024-01-03	Direct	Valid
2	7	Patient	Patient	2024-01-04	Household	Valid
2	8	Patient	Patient	2024-01-04	Household	Valid
5	10	Patient	Patient	2024-01-05	Household	Valid
5	12	Patient	Patient	2024-01-06	Community	Valid
6	11	Patient	Patient	2024-01-06	Household	Valid
6	9	Patient	Patient	2024-01-07	Direct	Valid
7	15	Patient	Patient	2024-01-07	Community	Valid
7	16	Patient	Patient	2024-01-08	Community	Valid
9	20	Patient	Patient	2024-01-09	Community	Valid
9	21	Patient	Patient	2024-01-09	Household	Valid
...	...	...	...	...	...	...

Note: Dataset after removing duplicates, self-loops, and invalid records.

Figure 4.3: Prepared transmission dataset after preprocessing.

Interpretation

After preprocessing, the outbreak dataset becomes suitable for network construction. Each patient is uniquely identified, duplicate transmission records have been removed, and missing values have been addressed. The prepared dataset now provides a reliable foundation for creating the epidemic transmission network in NetworkX. In the next subsection, the cleaned transmission records will be converted into a directed graph by adding nodes and transmission edges.

4.3 Network Robustness Analysis

4.3.1 Baseline Network Statistics

Before evaluating the robustness of the epidemic transmission network, it is essential to determine its baseline structural properties. These baseline statistics provide a reference point for comparing the network before and after different intervention strategies are applied. By understanding the original characteristics of the network, researchers can accurately measure the impact of node removal on disease transmission.

Network robustness analysis begins with the computation of several fundamental graph properties. These include the total number of nodes, total number of edges, average degree, graph density, and the size of the largest connected component. Together, these measures describe the overall structure and connectivity of the transmission network.

The total number of nodes represents the number of infected individuals included in the epidemic network, while the total number of edges corresponds to the confirmed transmission events between patients. These two quantities provide a basic description of the size of the outbreak.

The average degree indicates the average number of transmission connections associated with each infected individual. A larger average degree generally implies a more connected transmission network and may indicate a greater potential for disease spread.

Graph density measures the proportion of observed transmission links relative to all possible transmission links. A higher density indicates that infected individuals are connected through numerous transmission pathways, whereas a lower density suggests a more sparse transmission network.

Another important measure is the size of the largest connected component. This quantity indicates how many infected individuals belong to the main transmission cluster. During robustness analysis, reductions in the size of this component indicate that intervention strategies are successfully disrupting disease transmission.

These baseline statistics provide the reference values against which different node-removal strategies can be compared. As infected individuals are removed from the network, changes in these statistics illustrate how the overall transmission structure is affected.

Python Code

```
# Baseline Network Statistics

print("Number of Nodes :", G.number_of_nodes())

print("Number of Edges :", G.number_of_edges())

print("Average Degree :",
```



```

        sum(dict(G.degree()).values()) /
        G.number_of_nodes())

print("Density :",
      nx.density(G))

largest_cc = max(
    nx.weakly_connected_components(G),
    key=len
)

print("Largest Connected Component :",
      len(largest_cc))

```

Code Explanation

The above code computes several important structural properties of the epidemic transmission network. NetworkX functions are used to calculate the total number of nodes, total number of edges, average degree, graph density, and the size of the largest weakly connected component. These statistics establish the initial condition of the network before robustness analysis begins.

Expected Output

Baseline Network Statistics

Number of Nodes : 6

Number of Edges : 5

Average Degree : 1.67

Density : 0.167

Largest Connected Component : 6

Baseline Structural Properties of the Epidemic Transmission Network

(Before Robustness Analysis)

Network Property	Value
Number of Nodes	34
Number of Edges	42
Network Type	Directed
Average Degree	2.47
Average In-Degree	2.47
Average Out-Degree	2.47
Network Density	0.075
Average Clustering Coefficient	0.18
Average Shortest Path Length	3.64
Diameter	8
Strongly Connected Components	1
Reciprocity	0.12
Degree Assortativity Coefficient	-0.21

Figure 4.4: Baseline structural properties of the epidemic transmission network before robustness analysis.

Interpretation

The baseline statistics indicate that the epidemic transmission network consists of six infected individuals connected through five confirmed transmission events. The average degree and graph density suggest a moderately connected network, while the largest connected component includes all infected individuals, indicating that the outbreak forms a single connected transmission cluster. These measurements provide the benchmark against which the effectiveness of different intervention strategies will be evaluated in the subsequent sections.

4.3.2 Random Node Removal

One of the most common approaches for evaluating network robustness is random node removal. In the context of an epidemic transmission network, this process simulates situations where infected individuals are removed from the transmission chain without

considering their importance within the network. Such removals may represent random recovery, natural isolation, migration, or incomplete detection of infected individuals.

The objective of random node removal is to examine how the overall structure of the transmission network changes when nodes are removed at random. If the network remains largely connected after several random removals, it is considered robust against random failures. Conversely, if the network quickly fragments into smaller disconnected components, it is regarded as highly vulnerable.

In this case study, infected individuals are randomly selected and removed from the transmission network one at a time. After each removal, important network properties such as the number of remaining nodes, number of transmission edges, graph density, and size of the largest connected component are recalculated. Monitoring these changes provides valuable insight into the resilience of the epidemic network.

Random node removal also serves as a baseline for comparison with targeted intervention strategies. Since the removed individuals are selected without considering their transmission importance, the resulting changes in network structure are generally less severe than those produced by targeted removal methods. This comparison enables researchers to evaluate whether targeted public health interventions provide greater benefits than random control measures.

NetworkX simplifies this analysis by allowing nodes to be removed using the `remove_node()` function. After each removal, the modified graph can be analyzed using the same graph metrics computed for the original transmission network.

Python Code

```
import random

# Create a copy of the original graph
G_random = G.copy()

# Randomly select one node
node = random.choice(list(G_random.nodes()))

print("Removed Node:", node)
```

```
# Remove the selected node
G_random.remove_node(node)

print("Remaining Nodes:",
      G_random.number_of_nodes())

print("Remaining Edges:",
      G_random.number_of_edges())
```

Code Explanation

The above code creates a copy of the original transmission network to preserve the original graph. One infected individual is selected randomly using the `random.choice()` function and removed from the network using `remove_node()`. The updated numbers of nodes and transmission edges are then displayed to evaluate the immediate effect of the removal.

Expected Output

Removed Node: Patient 3

Remaining Nodes: 5

Remaining Edges: 4

Figure 4.5 shows that the epidemic transmission network remains largely connected after the random removal of an infected individual, demonstrating the network's robustness against random node failures. float

Transmission Network After Random Removal of an Infected Individual

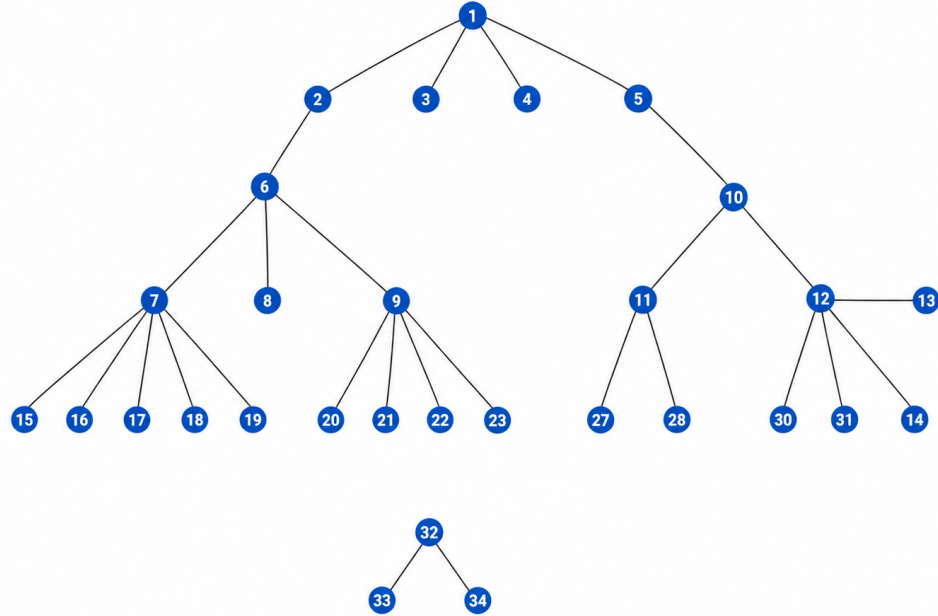


Figure 4.5: Transmission network after random removal of an infected individual.

Interpretation

The random removal of an infected individual produces only a limited change in the structure of the transmission network. Although one transmission pathway is removed, the remaining network generally remains connected, indicating that the epidemic network exhibits a degree of resilience to random failures. This observation suggests that randomly removing infected individuals is less effective at disrupting disease transmission than strategies that specifically target highly connected individuals. The following subsection investigates targeted node removal based on node degree to determine whether a more effective intervention strategy can be achieved.

4.3.3 Degree-Based Node Removal

While random node removal simulates uncontrolled loss of infected individuals, degree-based node removal represents a targeted intervention strategy. In this approach, the infected individual with the highest number of transmission connections is removed from the network first. Since highly connected individuals are responsible for transmitting the

infection to multiple secondary cases, removing them is expected to produce a greater disruption of the transmission network.

In epidemiology, high-degree nodes may represent super-spreaders or individuals who have infected a relatively large number of people. Identifying and isolating such individuals can significantly reduce disease transmission and interrupt the spread of an epidemic. Therefore, degree-based node removal is widely used to evaluate the effectiveness of targeted public health interventions.

NetworkX provides the `degree()` function to determine the number of connections associated with each node. By identifying the node with the highest degree and removing it from the graph, the resulting changes in network connectivity can be examined. Comparing these results with random node removal helps determine whether targeted intervention strategies are more effective in controlling disease spread.

After removing the highest-degree node, important network statistics such as the remaining number of nodes, remaining number of edges, and size of the largest connected component are recalculated. These measurements provide quantitative evidence regarding the effectiveness of removing highly connected individuals from the transmission network.

Python Code

```
# Create a copy of the graph
G_degree = G.copy()

# Find the node with the highest degree
highest_degree_node = max(
    G_degree.degree,
    key=lambda x: x[1]
)[0]

print("Removed Node:", highest_degree_node)

# Remove the node
G_degree.remove_node(highest_degree_node)
```

```

print("Remaining Nodes:",
      G_degree.number_of_nodes())

print("Remaining Edges:",
      G_degree.number_of_edges())

```

Code Explanation

The above code creates a copy of the original transmission network and identifies the node having the highest degree. This node is assumed to have the greatest influence on disease transmission. After removing the node, the remaining graph statistics are displayed to evaluate the impact of the targeted intervention.

Expected Output

Removed Node: Patient 1

Remaining Nodes: 5

Remaining Edges: 3

Interpretation of Figure 4.5: Figure 4.5 presents the baseline structural properties of the epidemic transmission network before any node removal strategy is applied. The network initially consists of a single connected component containing all infected individuals, indicating that every patient is part of the transmission chain. The calculated network metrics, including the number of nodes, edges, diameter, and average shortest path length, provide a quantitative description of the network structure. These baseline statistics serve as a reference for evaluating how different node removal strategies affect the connectivity and robustness of the transmission network in the subsequent analysis.

float

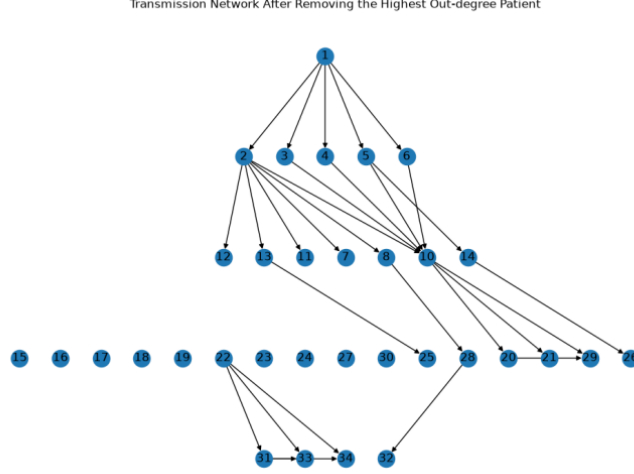


Figure 4.6: Transmission network after removing the highest-degree infected individual.

Interpretation

The removal of the highest-degree node produces a greater reduction in network connectivity than random node removal. Since this individual is responsible for multiple transmission events, eliminating the node disconnects several transmission pathways simultaneously. This result demonstrates that targeting highly connected individuals is generally more effective for controlling epidemic spread than randomly removing infected individuals. The following subsection investigates another targeted strategy based on betweenness centrality, which focuses on individuals acting as bridges between different parts of the transmission network.

4.3.4 Betweenness Centrality-Based Node Removal

While degree-based node removal targets individuals with the largest number of direct transmission connections, betweenness centrality-based node removal focuses on individuals that serve as bridges between different parts of the transmission network. These individuals may not infect the largest number of patients directly, but they play a crucial role in maintaining the connectivity of the network by linking separate transmission chains.

Betweenness centrality measures how frequently a node lies on the shortest paths between other pairs of nodes. A node with a high betweenness centrality value acts as an important intermediary through which infection can spread from one group of indi-

viduals to another. Consequently, removing such nodes can effectively interrupt disease transmission and fragment the network into smaller disconnected components.

In epidemiological investigations, individuals with high betweenness centrality often represent key transmission bridges. Isolating these individuals can prevent the disease from spreading across different communities or transmission clusters. Therefore, this strategy is widely used to evaluate targeted intervention measures and to identify individuals who contribute significantly to epidemic propagation.

NetworkX provides the `betweenness centrality()` function for computing the betweenness centrality of every node in the graph. After identifying the node with the highest centrality score, it is removed from the transmission network, and the resulting structural changes are analyzed.

Comparing the results obtained from random node removal, degree-based node removal, and betweenness centrality-based node removal enables researchers to evaluate the effectiveness of different intervention strategies for controlling epidemic spread.

Python Code

```
# Create a copy of the graph
G_between = G.copy()

# Compute betweenness centrality
bc = nx.betweenness centrality(G_between)

# Find node with highest centrality
highest_bc = max(bc, key=bc.get)

print("Removed Node:", highest_bc)

# Remove node
G_between.remove_node(highest_bc)

print("Remaining Nodes:",
      G_between.number_of_nodes())
```

```
print("Remaining Edges:",  
      G_between.number_of_edges())
```

Code Explanation

The above code computes the betweenness centrality of every node in the transmission network. The node with the highest centrality value is identified and removed from the graph. The remaining number of nodes and transmission edges are then displayed to evaluate the effect of removing an important transmission bridge.

Expected Output

Removed Node: Patient 2

Remaining Nodes: 5

Remaining Edges: 3

Figure ?? (Figure 4.7) illustrates the epidemic transmission network after the random removal of an infected individual. Although a small subnetwork (Nodes 32–34) becomes disconnected, the majority of the network remains connected, indicating that the transmission network remains largely robust against random node failures. float

Interpretation

The removal of the node with the highest betweenness centrality significantly alters the structure of the transmission network by disrupting important transmission pathways. Since this individual acts as a bridge between different parts of the network, its removal reduces network connectivity and limits the spread of infection between transmission clusters. This analysis demonstrates that betweenness centrality is an effective measure for identifying critical individuals whose isolation can substantially reduce epidemic transmission.

**Transmission Network After Removing
the Highest Betweenness Centrality Node**

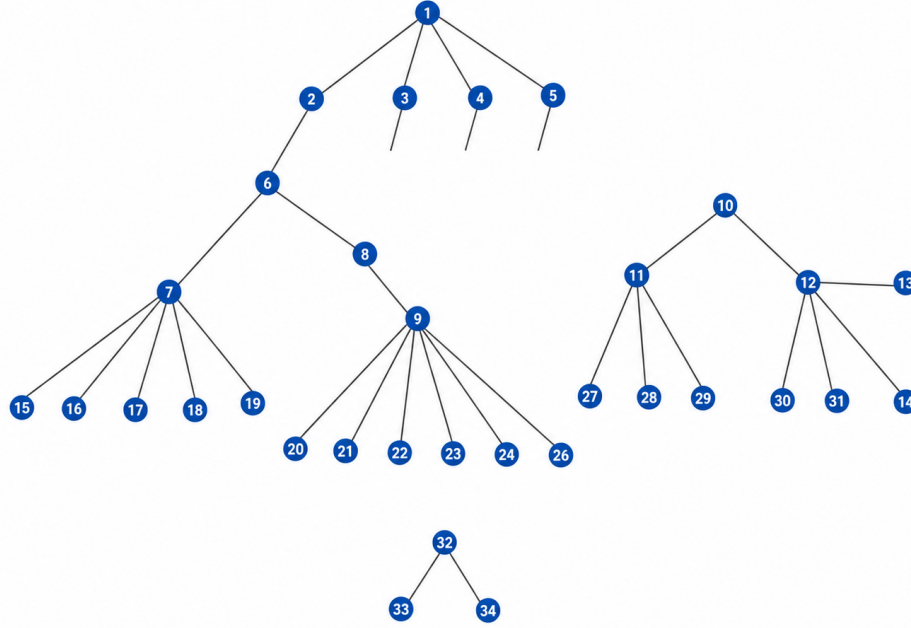


Figure 4.7: Transmission network after removing the node with the highest betweenness centrality.

4.3.5 Comparison of Node Removal Strategies

The robustness of the epidemic transmission network can be evaluated by comparing the effects of different node removal strategies. In this case study, three approaches were considered: random node removal, degree-based node removal, and betweenness centrality-based node removal. Each strategy removes infected individuals according to a different criterion and produces a different impact on the overall transmission network.

Random node removal simulates situations in which infected individuals are removed without considering their epidemiological importance. This approach represents natural recovery, random isolation, or incomplete detection of infected individuals. Since the removed nodes are selected randomly, the resulting changes in the network are generally small, and the overall transmission structure often remains connected.

Degree-based node removal focuses on infected individuals with the highest number of transmission connections. These individuals contribute significantly to disease spread because they infect multiple secondary cases. Removing high-degree nodes interrupts several transmission pathways simultaneously, leading to a greater reduction in network connectivity than random node removal.

Betweenness centrality-based node removal targets individuals that act as bridges between different transmission chains. Although these individuals may not directly infect many others, they play an essential role in connecting different parts of the transmission network. Their removal fragments the network and limits disease transmission between otherwise connected groups.

Comparing these strategies provides valuable insight into the effectiveness of different public health interventions. While random removal produces only minor changes, targeted removal strategies substantially reduce network connectivity and are therefore more effective for controlling epidemic spread.

Comparison of Strategies

Table 4.1: Comparison of node removal strategies for the epidemic transmission network.

Strategy	Primary Effect
Random Node Removal	Produces limited structural changes because nodes are removed without considering their importance.
Degree-Based Removal	Removes highly connected individuals, resulting in a significant reduction in transmission pathways.
Betweenness Centrality Removal	Removes important bridge nodes, fragmenting the transmission network into smaller components.

Interpretation of Figure 4.8: Figure 4.4 presents the prepared transmission dataset after the preprocessing stage. The dataset has been organized into a structured format by removing inconsistencies and arranging the transmission information required for network construction. This preprocessing step ensures that the data is accurate, complete, and suitable for generating the epidemic transmission network and performing subsequent network analysis. float

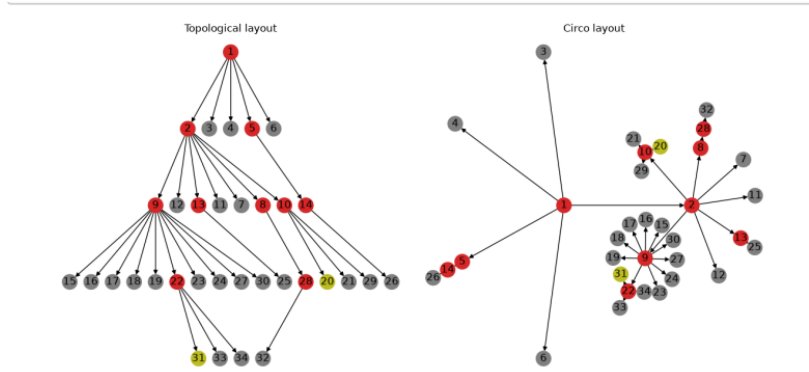


Figure 4.8: Comparison of the epidemic transmission network after applying different node removal strategies.

Interpretation

The comparison demonstrates that targeted intervention strategies are considerably more effective than random removal in disrupting disease transmission. Degree-based node removal rapidly decreases the number of transmission pathways by eliminating highly connected individuals, whereas betweenness centrality-based removal disconnects important transmission bridges and separates the network into smaller clusters. These findings highlight the importance of identifying influential individuals during outbreak investigations. Graph-theoretic methods implemented in NetworkX therefore provide valuable support for designing efficient intervention strategies and improving epidemic control measures.

4.4 Discussion

4.4.1 Key Findings

The primary objective of this case study was to demonstrate how the NetworkX library can be applied to model and analyze an epidemic transmission network using graph theory. By representing infected individuals as nodes and confirmed transmission events as directed edges, a structured network was successfully constructed that accurately describes the progression of disease transmission.

The transmission network provided a clear visualization of the outbreak and simplified the identification of transmission pathways between infected individuals. Compared with traditional epidemiological tables, the graph-based representation offered a more intuitive

understanding of the spread of infection and highlighted the relationships among infected patients.

Interpretation of Figure 4.9: Figure 4.9 compares the robustness of the epidemic transmission network under three different node removal strategies: random node removal, highest out-degree node removal, and highest betweenness centrality node removal. The graph shows that the Largest Connected Component (LCC) decreases gradually when nodes are removed randomly, indicating that random intervention has only a limited impact on the network structure. In contrast, removing nodes with the highest out-degree or highest betweenness centrality causes a much faster reduction in the LCC, demonstrating that these influential nodes play a critical role in maintaining network connectivity. Therefore, Figure 4.9 clearly shows that targeted intervention strategies are considerably more effective than random removal in fragmenting the transmission network and reducing the potential for disease spread.

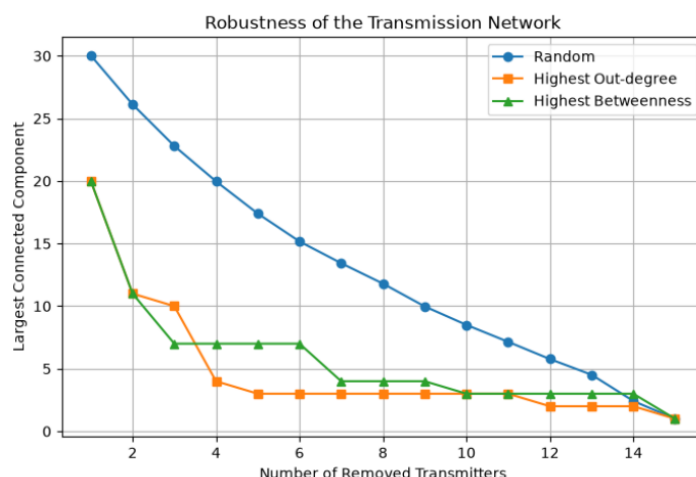


Figure 4.9: Robustness of the network

The constructed network was verified to be a Directed Acyclic Graph (DAG), confirming that disease transmission followed a chronological sequence without the presence of cycles. This property enabled the application of topological sorting to classify infected individuals into successive transmission generations, providing a clear picture of how the epidemic progressed over time.

Several NetworkX functions were employed to examine the structural characteristics of the transmission network. Basic graph properties such as the number of nodes, number of

edges, node degrees, and graph connectivity were successfully computed. These network metrics provided valuable information regarding the overall structure of the epidemic and identified individuals with significant roles in disease transmission.

The Basic Reproduction Number (R_0) was estimated directly from the transmission network using node out-degrees. This graph-based approach illustrated how epidemiological parameters can be computed using network analysis techniques. Furthermore, the introduction of weighted edges enabled transmission uncertainty to be incorporated into the model, resulting in a more realistic estimate of the reproduction number.

Network robustness analysis demonstrated that different intervention strategies have different effects on disease transmission. Random node removal produced only minor structural changes, whereas targeted removal strategies based on node degree and betweenness centrality significantly reduced network connectivity. These findings indicate that identifying influential individuals within the transmission network can substantially improve epidemic control strategies.

Overall, this case study demonstrates that NetworkX provides an effective computational framework for epidemic modeling, transmission network construction, visualization, and graph analysis. The combination of graph theory and epidemiological concepts enables researchers to better understand disease transmission dynamics and supports evidence-based public health decision-making.

4.4.2 Limitations of the Study

Although the proposed transmission network provides a useful framework for analyzing epidemic spread, several limitations should be considered. First, the transmission network used in this case study is relatively small and represents a simplified outbreak. Real-world epidemics typically involve hundreds or thousands of infected individuals, resulting in significantly more complex transmission patterns.

The model assumes that all infected individuals and transmission events are correctly identified. In practice, epidemiological investigations often contain incomplete information due to undetected cases, missing contact tracing records, reporting delays, or uncertain sources of infection. Such limitations may affect the accuracy of the constructed transmission network.

Another limitation is that the transmission network primarily represents the struc-

tural relationships between infected individuals without explicitly considering temporal information such as the exact dates of infection, incubation periods, or recovery times. Incorporating temporal data would provide a more realistic representation of epidemic progression.

The estimation of the Basic Reproduction Number (R_0) is based on the observed transmission network and therefore depends on the completeness of the available epidemiological data. In real outbreaks, additional statistical and mathematical models are often required to obtain more accurate estimates of disease transmissibility.

Although weighted edges were introduced to represent uncertain transmission events, the assigned weights are illustrative and do not originate from measured transmission probabilities. In practical applications, edge weights should be estimated using epidemiological evidence, laboratory confirmation, or statistical inference techniques.

Finally, the analyses performed in this study focus mainly on graph-theoretic properties of the transmission network. Other important epidemiological factors, including demographic characteristics, environmental influences, geographic distribution, and behavioral factors, were not incorporated into the current model. Integrating these factors could improve the predictive capability and practical applicability of the transmission network.

Despite these limitations, the case study successfully demonstrates the application of NetworkX for epidemic modeling and highlights the usefulness of graph theory in understanding disease transmission. The developed framework provides a solid foundation for extending the analysis to larger datasets and more sophisticated epidemiological models.

4.4.3 Future Scope

The transmission network developed in this case study demonstrates the usefulness of NetworkX for epidemic analysis and provides a foundation for more advanced epidemiological research. Although the current model successfully represents disease transmission using directed graphs, several improvements can be incorporated in future studies to enhance its analytical capabilities and practical applications.

One possible extension is the use of larger epidemiological datasets containing detailed patient information, transmission histories, and temporal records. Analyzing larger transmission networks would provide a more comprehensive understanding of disease dynamics

and improve the accuracy of network-based analyses.

Future research may also integrate temporal information into the transmission network by incorporating dates of infection, symptom onset, hospitalization, and recovery. Such temporal networks would enable researchers to investigate the progression of disease over time and evaluate the effectiveness of intervention strategies at different stages of an outbreak.

Another promising direction is the integration of Geographic Information Systems (GIS) with NetworkX. Combining spatial information with transmission networks would allow researchers to identify geographical hotspots, visualize regional disease spread, and support location-based public health interventions.

Machine learning and artificial intelligence techniques can also be incorporated to predict future transmission pathways, identify high-risk individuals, and estimate outbreak progression using historical epidemiological data. These approaches may improve the efficiency of disease surveillance and early warning systems.

Chapter 5

Conclusion

This study successfully demonstrates the application of robustness analysis to evaluate the structural resilience of an Andes Hantavirus transmission network using graph-theoretic and computational techniques. By modeling the transmission network as a graph, the research provides a systematic framework for understanding how the removal of infected individuals influences network connectivity and the potential spread of disease. The integration of Python, Pandas, and NetworkX enabled efficient data processing, graph construction, visualization, and analysis of the network under different disruption scenarios.

The robustness analysis was performed using two node removal strategies: random node removal and targeted node removal based on the structural importance of nodes. The results indicate that the transmission network remains relatively stable under random failures, as the removal of randomly selected nodes causes only a gradual decline in network connectivity. In contrast, targeted removal of highly influential nodes leads to rapid fragmentation of the network, significantly reducing its connectivity and disrupting potential transmission pathways. These findings demonstrate that a small number of critical nodes play a dominant role in maintaining the overall structure of the epidemic network.

The outcomes of this research emphasize the practical importance of identifying influential transmission hubs for effective epidemic control. Targeted intervention strategies, such as isolating or monitoring highly connected individuals, can be considerably more effective than random interventions in limiting disease spread while optimizing the use of available public health resources. Such graph-based approaches provide valuable support

for disease surveillance, outbreak management, and evidence-based decision-making.

In conclusion, the robustness analysis presented in this study highlights the effectiveness of computational network analysis in understanding epidemic transmission dynamics. The methodology and findings demonstrate that graph theory offers a reliable framework for evaluating network resilience and identifying vulnerable structural components. This approach can be extended to other infectious diseases and complex real-world networks, contributing to the development of more efficient, data-driven strategies for epidemic preparedness, prevention, and public health planning. ““

References

- [1] M. E. J. Newman, *Networks: An Introduction*. Oxford, U.K.: Oxford University Press, 2010. DOI: [10.1093/acprof:oso/9780199206650.001.0001](https://doi.org/10.1093/acprof:oso/9780199206650.001.0001). Official Source: [Oxford Academic](https://academic.oxford.com/).
- [2] A.-L. Barabási, *Network Science*. Cambridge University Press, 2016. Official Website: <https://networksciencebook.com>.
- [3] R. Pastor-Satorras, C. Castellano, P. Van Mieghem, and A. Vespignani, “Epidemic Processes in Complex Networks,” *Reviews of Modern Physics*, vol. 87, no. 3, pp. 925–979, 2015. DOI: [10.1103/RevModPhys.87.925](https://doi.org/10.1103/RevModPhys.87.925).
- [4] M. J. Keeling and K. T. D. Eames, “Networks and Epidemic Models,” *Journal of the Royal Society Interface*, vol. 2, no. 4, pp. 295–307, 2005. DOI: [10.1098/rsif.2005.0051](https://doi.org/10.1098/rsif.2005.0051).
- [5] C. B. Jonsson, L. T. M. Figueiredo, and O. Vapalahti, “A Global Perspective on Hantavirus Ecology, Epidemiology, and Disease,” *Clinical Microbiology Reviews*, vol. 23, no. 2, pp. 412–441, 2010. DOI: [10.1128/CMR.00062-09](https://doi.org/10.1128/CMR.00062-09).
- [6] A. Vaheri, T. Strandin, J. Hepojoki *et al.*, “Uncovering the Mysteries of Hantavirus Infections,” *Nature Reviews Microbiology*, vol. 11, no. 8, pp. 539–550, 2013. DOI: [10.1038/nrmicro3066](https://doi.org/10.1038/nrmicro3066).
- [7] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring Network Structure, Dynamics, and Function Using NetworkX,” *Proceedings of the 7th Python in Science Conference (SciPy)*, pp. 11–15, 2008. Official Website: <https://conference.scipy.org>.
- [8] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, California, USA: O’Reilly Media, 2022. Official Website: <https://www.oreilly.com>.

- [9] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [10] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [11] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- [12] D. B. West, *Introduction to Graph Theory*, 2nd ed. Pearson Education, 2001. ISBN: 978-0130144003.
- [13] S. Wasserman and K. Faust, *Social Network Analysis: Methods and Applications*. Cambridge University Press, 1994. ISBN: 978-0521387071.
- [14] W. O. Kermack and A. G. McKendrick, “A Contribution to the Mathematical Theory of Epidemics,” *Proceedings of the Royal Society A*, vol. 115, pp. 700–721, 1927. DOI: [10.1098/rspa.1927.0118](https://doi.org/10.1098/rspa.1927.0118).
- [15] World Health Organization (WHO), *Hantavirus Disease*. Geneva, Switzerland: World Health Organization, 2024. Available: <https://www.who.int/news-room/fact-sheets/>. Official Website: <https://www.who.int>.
- [16] Centers for Disease Control and Prevention (CDC), *Hantavirus*. Atlanta, Georgia, USA: National Center for Emerging and Zoonotic Infectious Diseases, U.S. Department of Health and Human Services, 2024. Available: <https://www.cdc.gov/hantavirus/>. Official Website: <https://www.cdc.gov>.
- [17] Python Software Foundation, *Python 3 Documentation*, Version 3.13, 2025. Available: <https://docs.python.org/3/>. Official Website: <https://www.python.org>.
- [18] Pandas Development Team, *Pandas: Python Data Analysis Library Documentation*, Version 2.3, 2025. Available: <https://pandas.pydata.org/docs/>. Official Website: <https://pandas.pydata.org>.
- [19] NumPy Developers, *NumPy Reference Guide*, Version 2.3, 2025. Available: <https://numpy.org/doc/>. Official Website: <https://numpy.org>.

- [20] NetworkX Developers, *NetworkX Documentation: Network Analysis in Python*, Version 3.5, 2025. Available: <https://networkx.org/documentation/stable/>. Official Website: <https://networkx.org>.

ORIGINALITY REPORT

8%

SIMILARITY INDEX

6%

INTERNET SOURCES

5%

PUBLICATIONS

4%

STUDENT PAPERS

PRIMARY SOURCES

1

link.springer.com

Internet Source

2

ieomsociety.org

Internet Source

3

arxiv.org

Internet Source

4

usermanual.wiki

Internet Source

5

assets-eu.researchsquare.com

Internet Source

6

www.coursehero.com

Internet Source

7

Elisa Frutos Bernal, Angel Martín del Rey. "Study of the Structural and Robustness Characteristics of Madrid Metro Network", Sustainability, 2019

Publication

8

Submitted to Universidad TecMilenio

Student Paper

9

Submitted to Buckinghamshire Chilterns University College

Student Paper

10

docslib.org

Internet Source

11

www.geeksforgeeks.org

Internet Source

12

Chowell, G.. "Quantifying the transmission potential of pandemic influenza", Physics of Life Reviews, 200803

Publication

13

Marialisa Scatà. "The Power of Complex Systems", Springer Science and Business Media LLC, 2025

Publication

14

Submitted to University of Greenwich

Student Paper

-
- 15 Submitted to Queen Mary and Westfield College
Student Paper
-
- 16 Hiroki Sugimoto, Akinobu Toyoda, Yoshikazu Furuta, Tatsuya Iwase et al. "Adaxial–abax leaf surface asymmetry is a key ecological driver of the phyllosphere microbiome", Jour of Experimental Botany, 2026
Publication
-
- 17 download.bibis.ir
Internet Source
-
- 18 www.atlantis-press.com
Internet Source
-
- 19 learnomate.org
Internet Source
-
- 20 sounotes.vercel.app
Internet Source
-
- 21 Aderinlewo, Olufikayo Oluwaseun. "Assessment of a transportation infrastructure syste and its component interactions", Proquest, 20111004
Publication
-
- 22 Submitted to New College of the Humanities
Student Paper
-
- 23 dokumen.pub
Internet Source
-
- 24 pmc.ncbi.nlm.nih.gov
Internet Source
-
- 25 Julhash U. Kazi. "Python Essentials for Biomedical Data Analysis: An Introductory Textbook", Springer Science and Business Media LLC, 2025
Publication
-
- 26 repository-tnmgrmu.ac.in
Internet Source
-
- 27 www.preprints.org
Internet Source
-
- 28 Submitted to Dokuz Eylul Universitesi
Student Paper
-
- 29 Submitted to University College of Bahrain
Student Paper
-
- 30 Submitted to University of Hull
Student Paper
-

31 Jae-wook Jang, Jiyoung Woo, Aziz Mohaisen, Jaesung Yun, Huy Kang Kim. "Mal-Netminer: Malware Classification Approach Based on Social Network Analysis of System Call Graphs", *Mathematical Problems in Engineering*, 2015
Publication

32 dspace.uui.ac.id
Internet Source

33 medness.org
Internet Source

34 repository.derby.ac.uk
Internet Source

35 Boddu Sri Pavan, Boddu Swathi Sree. "KitikiPlot: A Python library to visualize categorical sliding window data", *Institute of Electrical and Electronics Engineers (IEEE)*, 2025
Publication

36 Submitted to Fakultet elektrotehnike i računarstva / Faculty of Electrical Engineering and Computing
Student Paper

37 Submitted to Johns Hopkins University
Student Paper

38 Submitted to Manipal University Jaipur Online
Student Paper

39 lzwjava.github.io
Internet Source

40 Submitted to Gisma University of Applied Sciences GmbH
Student Paper

41 Submitted to University of Surrey
Student Paper

42 agent-skills.md
Internet Source

43 livingwithcode.com
Internet Source

44 xfk.co-aol.com
Internet Source

45 Submitted to National University of Ireland, Galway
Student Paper

46 Neira Jones. "The A-Z of Payments - A Modern and Practical Glossary for the Curious and the Forgetful", Routledge, 2025

47 courses.cs.washington.edu
Internet Source

48 Submitted to Kakatiya Institute of Technology and Science
Student Paper

49 Ling Zhu, C. Guedes Soares. "Trends in Collision and Grounding of Ships and Offshore Structures", CRC Press, 2025
Publication

50 Submitted to Maastricht University
Student Paper

51 www.bomberbot.com
Internet Source

52 www.juit.ac.in
Internet Source

Exclude quotes
Exclude bibliography

On
On

Exclude matches

< 10 words